Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a technique that combines a large language model with an external knowledge source. Instead of relying only on what the model memorised during training, a RAG system first retrieves relevant passages from a document collection and then uses them as context to generate an answer.

RAG reduces hallucinations because the answer is grounded in real documents. It also makes it easy to update knowledge: you only need to add new documents to the database, and there is no need to retrain the model.

A typical RAG application has two phases. The first phase is ingestion, where documents are loaded, cleaned, split into chunks, converted into embeddings and stored in a vector database. The second phase is retrieval, where the user question is embedded and the most similar chunks are fetched and passed to the language model.

The Ingestion Pipeline

Ingestion is the foundation of every RAG chatbot. If the data entering the database is messy, the answers coming out will be messy too. The diagram below shows the full flow that we will follow in this project.

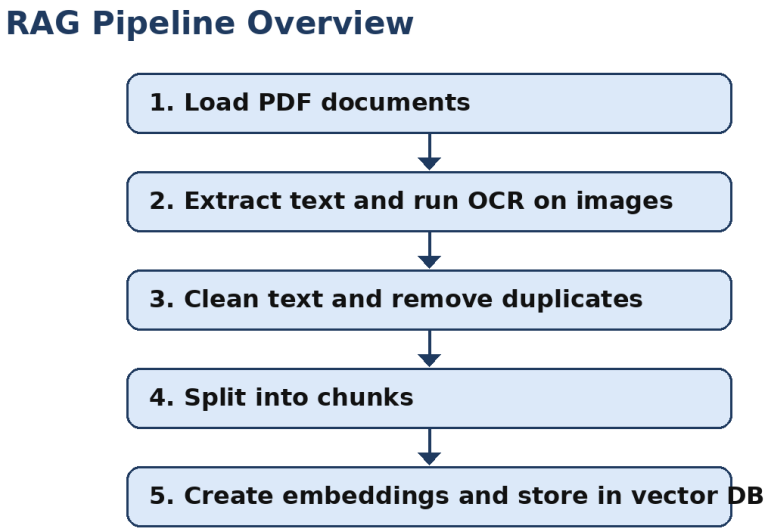


Figure 1: Overall RAG pipeline (text inside this figure is only available through OCR).

Each step has a clear responsibility. Extraction gets raw content from the file, cleaning normalises it, chunking prepares it for search and embedding turns text into numbers that can be compared.

Extracting Text from PDF Files

Many PDF files store their text in a text layer. Python libraries such as pypdf can read this layer page by page and return the text as a string. This is fast and accurate for digitally created documents.

However, pypdf cannot read text that is part of an image. Scanned pages, screenshots, charts and infographics are stored as pictures, so the extractor will return nothing for them. That is why we need OCR as a second extraction method.

When extracting, keep track of the page number for every piece of text. This metadata will later allow the chatbot to tell the user exactly which page an answer came from.

Some PDFs contain headers and footers that repeat on every page. These repeated lines add noise and duplicates to the data and should be removed during the cleaning stage.

Embedding Models

An embedding model converts a piece of text into a vector of numbers. Texts with similar meaning have vectors that are close to each other. Google provides several embedding models through the Gemini API, and the table below compares a few popular choices.

Embedding Model Comparison

Model	Dimensions	Max Tokens
text-embedding-004	768	2048
gemini-embedding-001	3072	2048
all-MiniLM-L6-v2	384	256

Figure 2: Embedding model comparison table.

Higher dimensions can capture more detail, but they also need more storage and make search slightly slower. For a first project, a 768-dimension model is a good balance.

Practical Tips

The notice below was added as an image on purpose. It simulates a scanned page or a screenshot that a normal text extractor cannot read.

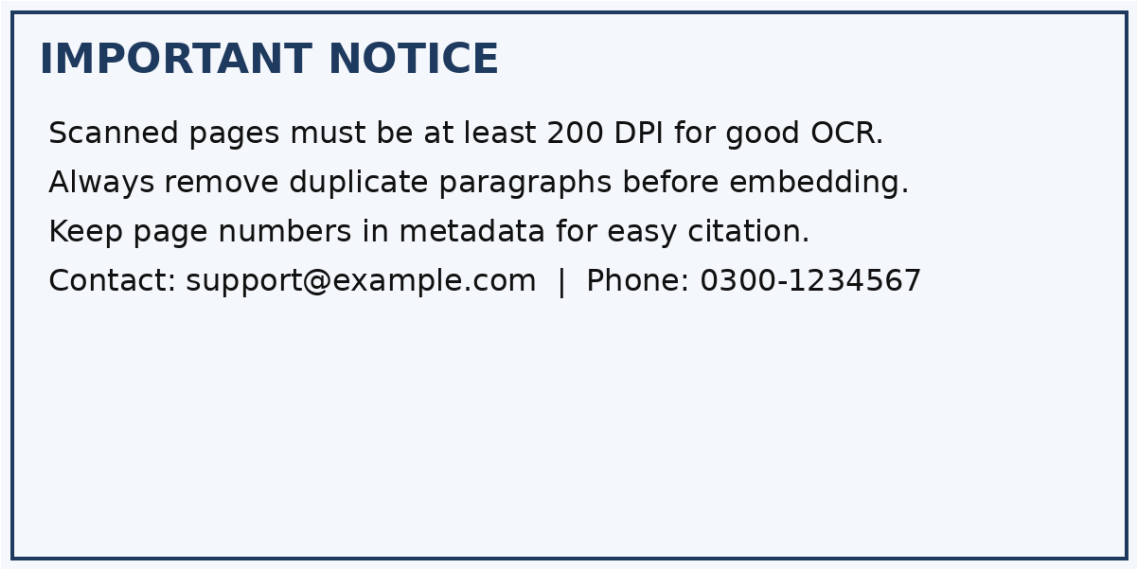


Figure 3: Practical tips (image only).

Always remove duplicate paragraphs before embedding.

Optical Character Recognition (OCR)

OCR is the process of converting images of text into machine-readable text. Tesseract is a free and widely used OCR engine, and it can be called from Python using the pytesseract package.

For best results, images should be clear, high resolution and have good contrast. Converting the image to grayscale and increasing its size before OCR often improves accuracy.

OCR is never perfect. It may confuse similar characters such as the letter O and the number 0. This is one more reason why a cleaning step with a language model is valuable: it can fix obvious mistakes and restore proper formatting.

Retrieval-Augmented Generation (RAG) is a technique that combines a large language model with an external knowledge source.

Chunking the Text

Language models and embedding models can only process a limited amount of text at once, and small focused chunks give more precise search results. Therefore, long documents must be split into chunks before embedding.

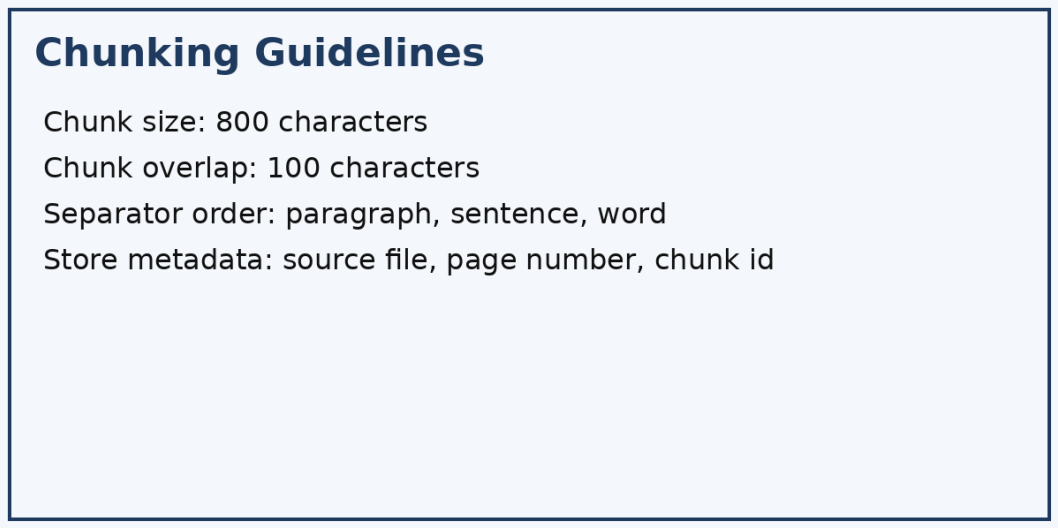


Figure 4: Recommended chunking settings.

Overlap between chunks keeps sentences that fall on a boundary from losing their context.

Cleaning and Deduplication

After extraction, the combined text from pypdf and OCR usually contains problems: extra spaces, broken lines, repeated headers and footers, and sometimes the same information appearing twice because it exists both in the text layer and in an image.

A simple rule-based pass can remove extra whitespace and exact duplicates. A language model can then handle the harder cases, such as near-duplicate sentences, OCR mistakes and inconsistent formatting.

Important: when using an LLM for cleaning, instruct it never to add new facts and never to summarise. Its job is only to tidy the text, not to change its meaning.

An embedding model converts a piece of text into a vector of numbers.

Storing Vectors

Once embeddings are created they must be stored together with the original chunk text and its metadata. A vector database is designed for this job and can quickly find the nearest vectors to a query.

Vector Database Options

Database	Type	Best For
ChromaDB	Local	Prototypes
FAISS	Local	Fast search
Pinecone	Cloud	Production

Figure 5: Common vector database options.

For learning and prototyping, a local database such as ChromaDB is the easiest to start with because it needs no account and no server.

Summary and Next Steps

In this document we covered the full ingestion flow of a RAG chatbot: extract text with pypdf, read image text with OCR, combine and clean the data, create embeddings with a Google embedding model and save everything in a vector database.

Once ingestion works correctly, the next phase is retrieval. The user question is embedded, the most relevant chunks are fetched from the database, and the language model produces a final answer using those chunks as context.

Good luck with your project! For questions, contact support@example.com.